

LibMapaStatic

Librería de mapas por teselas para Qt — arquitectura, clases y herramientas

Documentación técnica · Qt 5.14 / 5.15 / 6.x (MinGW · MSVC · GCC)

Núcleo de teselas RMaps/XYZ · Widget de mapa (QCustomPlot) · Herramientas de línea de comandos y con mapa

Índice

1. Visión general — qué es y qué resuelve
2. Arquitectura por capas
3. Tipos y modelos de datos
4. Módulo **geo** — geodesia y teselas
5. Módulo **db** — SQLite
6. Módulo **tiles** — motor de teselas
7. Módulo **widget** — el mapa en pantalla
8. Módulo **io** — ficheros .geo
9. Flujos de interacción entre objetos
0. Codificación de teselas (la matemática)
1. Herramientas desarrolladas
2. Motor de descarga `TileFiller` y auto-freno
3. Decisiones de diseño e historia
4. Cómo compilar y usar

1 · Visión general

LibMapaStatic es una librería en C++/Qt para mostrar mapas basados en **teselas** (imágenes de 256×256 px organizadas por zoom/x/y) guardadas en bases de datos **SQLite** del tipo *RMaps/XYZ*, y para dibujar encima entidades vectoriales (puntos, líneas, polígonos) y **objetivos móviles** en tiempo real. Nace de reescribir una aplicación previa («EstacionTerrena») separando con claridad el **núcleo** (datos, geodesia, teselas) del **dibujo**, de modo que el núcleo se pueda probar sin ventanas.

Qué hace, en una frase por área:

- **Teselas base:** abre una o varias BD de teselas (OSM, satélite...), las cachea y las pinta fluido según navegas.
- **Entidades:** dibuja y edita puntos/líneas/polígonos en capas, con guardado/carga en SQLite y carga de ficheros `.geo`.
- **Objetivos móviles:** cientos de móviles con traza y etiqueta, actualizables varias veces por segundo, en una capa que se repinta sola.
- **Herramientas:** sondar una BD (`probe_db`), rasterizar vectores a teselas (`geo_to_tiles`), y **descargar/rellenar** teselas de una fuente XYZ (`fill_tiles` consola y `fill_map` con mapa).

El diseño evita los defectos del código original: jerarquías con punteros crudos y `void*`, macros de zoom dispersas, y una clase de dibujo que exponía 300 KB de cabecera de `QCustomPlot` a todo el que la usara. Aquí la identidad de cada entidad es un **entero**, el mapeo de zoom vive en **datos** (`datasets.json`), y `QCustomPlot` queda **oculto** tras un PIMPL.

2 · Arquitectura por capas

El código se organiza en dos bibliotecas estáticas y un conjunto de aplicaciones. El **núcleo** no depende de Widgets ni de QCustomPlot; el **widget** añade el mapa visual. Las herramientas usan una u otra según necesiten pantalla.

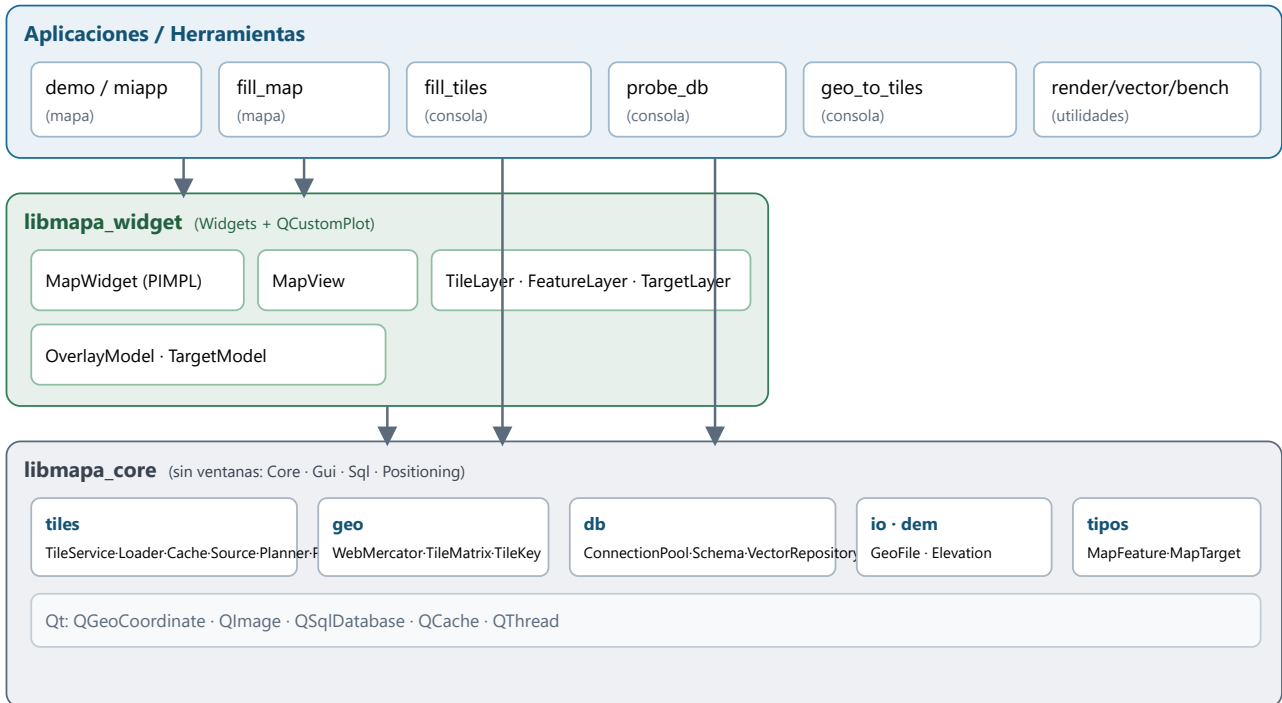


Figura 1 — Capas: las apps usan el widget o directamente el núcleo; el widget se apoya en el núcleo; el núcleo solo depende de Qt.

Motivo de la separación: el núcleo (teselas, geodesia, BD) se compila y prueba **sin QCustomPlot ni Widgets**. Así 14 de los 17 tests automáticos validan la lógica sin QCustomPlot ni ventanas, y quien solo quiera leer teselas no arrastra el motor de dibujo.

3 · Tipos y modelos de datos

Antes que las clases con lógica, conviene conocer los **tipos de valor** que se pasan por toda la librería. Son *structs* simples (sin herencia), identificados por un entero cuando hace falta.

Tipo	Rol
TileKey	Identidad de una tesela: {z, x, y} lógicos. Se usa como clave del caché (con qHash).
TileDataset	Todo lo que se sabe de una BD de teselas: ruta, tabla, mapeo de zoom (zFactor/zOffset), esquema (XYZ/TMS), columnas, hasSColumn/sValue , min/max/recommended zoom, relleno típico. Se lee de datasets.json .
MapFeature	Entidad vectorial: geometría multi-parte (puntos/líneas/polígonos), estilo (FeatureStyle), etiqueta y «tipo de dominio». Identidad = qint64 .
MapTarget	Objetivo móvil: posición, rumbo, color, etiqueta, visibilidad de traza. Pensado para tiempo real.
LayerInfo / BaseLayerInfo	Descriptor de una capa (de entidades / base) para poblar menús.
GeoPath	Un trazado leído de un fichero .geo (lista de coordenadas + si cierra).
Measurement / MapPoint / MapVehicle...	Tipos auxiliares de la API pública (medición, puntos, vehículos, rutas).

Regla de diseño: la identidad es un entero que devuelve la librería al añadir el objeto, no un puntero a una jerarquía. Esto elimina los QList<void*> y castes del código original.

4 · Módulo `geo` — geodesia y teselas

Convierte entre coordenadas geográficas (lat/lon) y el sistema de teselas Web Mercator. Es matemática pura, sin estado ni Qt más allá de `QGeoCoordinate`.

WebMercator `geo/WebMercator.h`

Función	Qué hace
<code>forward(coord) → QPointF</code>	Lat/lon → metros Web Mercator (EPSG:3857).
<code>inverse(x,y) → QGeoCoordinate</code>	Metros → lat/lon (la usa <code>geo_to_tiles</code> al leer <code>.xyz</code>).
<code>clampLatitude(lat)</code>	Acota la latitud al máximo de Mercator ($\sim \pm 85,05^\circ$) para que la proyección no se dispare.

TileMatrix `geo/TileMatrix.h`

El corazón geométrico: relaciona lat/lon con índices de tesela a cada zoom. Todo *static*.

Función	Qué hace
<code>tilesPerSide(z)</code>	2^z : cuántas teselas por lado tiene el mundo a ese zoom.
<code>longitudeToTileX / latitudeToTileY</code>	Coordenada → índice de tesela (fraccionario) en X / Y.
<code>tileXToLongitude / tileYToLatitude</code>	La inversa: índice de tesela → coordenada del borde.
<code>tileAt(coord, z) → TileKey</code>	Qué tesela contiene una coordenada a un zoom.
<code>tileBounds / tileNorthWest / tileSouthEast</code>	Rectángulo geográfico que cubre una tesela.
<code>rangeFor(nw, se, z, margen)</code>	Rango de teselas <code>[xMin..xMax]×[yMin..yMax]</code> que cubre un rectángulo — la base para pedir el viewport.
<code>toStorageY / fromStorageY(y,z,esquema)</code>	Convierte la Y entre XYZ (origen arriba) y TMS (origen abajo). Involutiva.
<code>latitudeToAxisY / axisYToLatitude</code>	Lat ↔ «grados de Mercator» del eje lineal de <code>QCustomPlot</code> (imprescindible para overlays).
<code>bestZoomFor(span, anchoPx)</code>	Elige el zoom natural para un ancho de mundo visible.

TileKey `tiles/TileKey.h`

Struct `{z,x,y}` con `operator==` y `qHash` para servir de clave en el caché. En Qt 6 el `qHash` se adapta (guarda de versión legítima).

5 · Módulo `db` — SQLite

SQLiteConnectionPool `db/SQLiteConnectionPool.h`

Da a cada hilo su propia conexión SQLite (Qt exige que una `QSqlDatabase` no se comparta entre hilos). Reutiliza conexiones por (fichero, hilo) y las nombra de forma única.

Función	Qué hace
<code>connectionFor(id, ruta)</code>	Devuelve/crea la conexión para ese fichero en el hilo actual.
<code>openConnectionCount()</code>	Cuántas conexiones vivas hay (diagnóstico y tests).

Transaction `db/Transaction.h`

Envoltura RAII de `BEGIN/COMMIT/ROLLBACK`: `commit()` confirma; si el objeto muere sin confirmar, hace *rollback*. Evita transacciones a medias.

Schema `db/Schema.h`

Crea/verifica el esquema de la BD de **entidades** (capas, geometrías, estilos) que usa `VectorRepository`.

VectorRepository `db/VectorRepository.h` · `QObject`

Persiste entidades vectoriales y capas en SQLite: guardar/cargar todo el mapa de entidades. Emite `errorOccurred`. Lo usa `MapWidget` en `saveFeaturesTo()/loadFeaturesFrom()`. Con `MapConfig.featuresDbFile` (o `setFeaturesDbFile()`) el widget hace **persistencia automática**: carga al abrir y guarda solo lo dibujado/editado/borrado con un **antirebote** (~500 ms) que agrupa las ráfagas — volcado completo, reutilizando `saveFeaturesTo`. Los objetivos móviles no se persisten.

6 · Módulo `tiles` — motor de teselas

Es el subsistema más grande: abrir BD de teselas, decidir qué pedir, leerlas en un hilo aparte, cachearlas y entregar un «plan de dibujo». Todo alrededor del `TileDataset`, que describe cada BD.

`TileDataset` `tiles/TileDataset.h`

Descriptor de una BD. Sustituye a los `#define` invertidos y parches de zoom del código original. Métodos clave:

Función	Qué hace
<code>storedZ(zLógico)</code>	$zFactor \cdot z + zOffset$: el valor que va en la columna <code>z</code> de la BD.
<code>logicalZ(zGuardado)</code>	La inversa.
<code>effectiveBaseZoom()</code>	Nivel de fondo garantizado, acotado al rango real.
<code>zoomInRange</code> / <code>zoomRecommended</code>	Si un zoom existe / si conviene ofrecerlo en la interfaz.
<code>fromJson</code> / <code>toJson</code>	Lee/escribe el descriptor en <code>datasets.json</code> .

`ITileSource` / `RMapsTileSource` `tiles/RMapsTileSource.h`

`ITileSource` es la interfaz «dame la imagen de esta `TileKey`». `RMapsTileSource` la implementa sobre una BD SQLite RMaps/XYZ: traduce el `TileKey` lógico a la fila real (con `storedZ`, la `Y` del esquema y la columna `s` si existe) y devuelve el BLOB de imagen.

Función	Qué hace
<code>open()</code>	Abre la BD y prepara las consultas (resuelve nombres de columna reales).
<code>fetch(key)</code>	Una tesela: <code>SELECT image WHERE z=storedZ AND x AND y [AND s]</code> .
<code>fetchRange(z, rango)</code>	Muchas teselas de un golpe (un viewport).
<code>availableInRange(...)</code>	Qué teselas existen (sin traer la imagen), para saber huecos.

`TileCache` `tiles/TileCache.h`

Caché LRU de imágenes ya decodificadas (basada en `QCache`). Guarda el coste en **KiB** (no bytes, para no desbordar el `int` con 2 GiB), tiene un área **anclada** (el nivel de fondo, que no se expulsa) y una lista de teselas «que no existen» para no volver a pedir las.

Función	Qué hace
<code>take(key)</code>	Copia (compartida) de la imagen si está; si no, cuenta un fallo.
<code>insert</code> / <code>insertImage</code>	Mete una tesela (decodificando o ya decodificada), con o sin anclar.
<code>markMissing</code> / <code>isKnownMissing</code>	Marca/consulta que una tesela no existe en la BD.
<code>countCached(...)</code>	Cuántas del rango están «resueltas» (en caché o marcadas ausentes).
<code>clear()</code> / <code>setMaxBytes()</code>	Vaciar (p. ej. tras <code>reloadBaseLayer</code>) / cambiar el presupuesto.

`TileLoader` `tiles/TileLoader.h` · `QObject` (hilo)

Lee teselas **en un hilo aparte** para no bloquear la interfaz. Recibe `TileRequest` (un rango + zoom + id), consulta la `ITileSource`, e informa con `LoadedTile`. El id de petición permite descartar respuestas viejas

cuando el usuario ya se movió.

TilePlanner `tiles/TilePlanner.h`

Calcula el **plan de dibujo**: qué teselas y con qué escala hay que pintar para cubrir el viewport, incluida la «escalera de respaldo» (usar teselas de zoom vecino mientras llega el detalle). Devuelve `TileDrawItems`.

TileService `tiles/TileService.h · QObject`

La **fachada** del módulo: orquesta dataset activo + caché + loader + planner. Es lo que usa el widget.

Función	Qué hace
<code>start(datasets, cacheMiB)</code>	Arranca con la lista de datasets y el presupuesto de caché.
<code>loadDatasets(json) [static]</code>	Lee <code>datasets.json</code> → lista de <code>TileDataset</code> .
<code>setActiveDataset(id)</code>	Cambia de capa base (osm↔satelital↔...).
<code>requestViewport(nw, se, z, margen)</code>	Pide al loader las teselas visibles (con antirebote).
<code>planFor(nw, se, z)</code>	Plan de dibujo con lo que haya en caché.
<code>cache()</code>	Acceso al <code>TileCache</code> (p. ej. para vaciarlo).
<code>signal tilesReady(id, z, nuevas)</code>	Llegaron teselas nuevas → repintar.

TileDatasetProbe `tiles/TileDatasetProbe.h`

Sonda: abre una BD «a ciegas» y **deduce** su configuración (mapeo de zoom, esquema, relleno, bbox) analizando qué hay en la tabla. Es el cerebro de `probe_db`. Devuelve `ProbeResult` con `ZoomLevelStats` por nivel.

7 · Módulo `widget` — el mapa en pantalla

Convierte el motor de teselas en un `QWidget` embebible. El motor de dibujo es `QCustomPlot`, pero queda oculto.

MapWidget `include/libmapa/MapWidget.h · QWidget (PIMPL)`

La **cara pública** de toda la librería. No expone `QCustomPlot`: por dentro tiene un `Impl` con el `TileService` y el `MapView`. Agrupa toda la API que usa la aplicación:

Grupo	Funciones (ejemplos)
Capa base	<code>availableBaseLayers · baseLayerId · setBaseLayerId · reloadBaseLayer</code>
Navegación	<code>center/setCenter · zoom/setZoom · zoomIn/Out · fitBounds · visibleNorthWest/SouthEast</code>
Capas de entidades	<code>addFeatureLayer · setFeatureLayerVisible · setFeatureLayerZOrder</code>
Entidades	<code>addFeature · updateFeature · removeFeature · features · loadGeoAsLayer</code>
Edición geometría	<code>moveVertex · insertVertex · removeVertex · moveFeature · undo/redo</code>
Paquete de datos	<code>MapConfig::dataDir · packageInfo · dataWarnings</code>
Persistencia	<code>saveFeaturesTo · loadFeaturesFrom · setFeaturesDbFile · saveFeaturesNow</code>
Objetivos móviles	<code>addTarget · updateTarget · setTargetLabel · setTargetTrailLength</code>
Herramientas / ratón	<code>setActiveTool (Measure, AreaZoom, SelectArea, Draw..., EditFeature) · clearAreaSelection</code>
Conversión	<code>toAxisCoords · fromAxisCoords</code>
Señales	<code>zoomChanged · centerChanged · areaSelected · clicked · featureCreated...</code>

MapView `widget/MapView.h · QCustomPlot`

El lienzo real: hereda de `QCustomPlot`, gestiona el ratón (arrastrar, rueda, herramientas), traduce píxeles↔coordenadas y coordina las capas. `requestVisibleTiles()` pide teselas del viewport; `refreshPlan()` rehace el plan y repinta. Aquí vive el rectángulo de `SelectArea` que se añadió para la descarga.

Capas de dibujo (`QCPLayerable`)

Clase	Rol
TileLayer	Pinta las teselas del plan (con la escalera de respaldo). Lee del <code>TileCache</code> .
FeatureLayer	Pinta las entidades vectoriales de una capa (según <code>OverlayModel</code>) y sus tiradores al editar.
TargetLayer	Pinta los objetivos móviles y su traza. Se repinta sola en modo <code>BUFFERED</code> sin rehacer teselas.
CoverageLayer	Mancha de cobertura : qué zonas de un zoom objetivo ya están en la BD, agregadas a una rejilla gruesa y coloreadas por completitud (ámbar→verde). Fija: se ve aunque mires a otro zoom.

Modelos (`QObject`, sin dibujo)

Clase	Rol
-------	-----

OverlayModel	Guarda las entidades y capas, la selección y el historial de deshacer/rehacer . Separado del dibujo para poder probarlo sin ventanas.
TargetModel	Guarda los objetivos y su traza (acotada). Operaciones $O(1)$ por objetivo para el tiempo real; emite <code>changed()</code> .

8 · Módulo `io` — ficheros `.geo` y paquete de datos

GeoFile `include/libmapa/GeoFile.h` lee/escribe ficheros `.geo` / `.xyz` : listas de coordenadas donde `0,0` separa trazados. Un fichero puede traer varios trazados (aguas jurisdiccionales, corredores, divisiones).

`MapWidget::loadGeoAsLayer()` lo carga como **una entidad multi-parte**. También lo usa `geo_to_tiles` para rasterizar.

DataPackage `io/DataPackage.h` · público: `DataPackageInfo`

El producto final trabaja **sin conexión**: todos sus datos viajan en un **paquete**, una carpeta con un manifiesto `mapa.json` (formato `libmapa-package`, versión 2) y rutas **relativas** a ella. `DataPackage::load()` lo lee (carpeta o fichero), valida formato y versión, resuelve las rutas a absolutas y anota como **aviso**, sin fallar, lo que falte. Un `datasets.json` de la versión 1 se acepta como paquete mínimo.

<code>package</code>	id, nombre, versión de los datos, fecha, zona (<code>bounds</code>) y atribución → <code>MapWidget::packageInfo()</code> .
<code>start</code>	Capa, centro y zoom de arranque.
<code>datasets</code>	Capas base (mismo formato que <code>datasets.json</code> ; lo omitido toma su valor por defecto).
<code>elevation</code>	<code>file</code> (BD <code>.sqlitedb</code>) o <code>dir</code> (carpeta <code>.hgt</code>).
<code>overlays</code>	Capas vectoriales fijas <code>.geo</code> con estilo: bloqueadas y excluidas del guardado de entidades (<code>fixedLayers</code>), para no duplicarse en cada arranque.
<code>features</code>	BD de entidades del usuario. Relativa → <code>AppDataLocation/<package.id>/</code> (escribible), nunca la carpeta del paquete; <code>seed</code> se copia la primera vez.

`MapConfig::dataDir` es el único ajuste necesario; cualquier campo de `MapConfig` rellenado a mano tiene prioridad sobre el paquete.

PackageCheck `io/PackageCheck.h`

Comprueba que un paquete funcionará **sin conexión**: lo que falte no se podrá descargar después. Cada hallazgo lleva su gravedad: **Error** si una parte del mapa no se dibujará, **Warning** si funciona pero algo falta o no es portable, **Info** para el informe. No escribe nada (todo en solo lectura; la BD de entidades del usuario ni se abre, porque abrirla podría migrar su esquema).

<code>Qt</code>	Driver <code>SQLITE</code> disponible (sin él no abre nada).
<code>Capas base</code>	El fichero existe, abre como base de teselas, y una tesela real del nivel de fondo se decodifica : detecta el plugin de imagen que falta al desplegar (<code>qjpeg</code>) y lo nombra.
<code>Cobertura</code>	Solo en modo completo: por cada zoom, teselas presentes / esperadas dentro de <code>bounds</code> , con <code>RMapsTileSource::countInRange</code> (un <code>COUNT(*)</code> que usa el índice: ~0,2 s para 7 GB).
<code>Elevación / capas fijas</code>	La BD tiene <code>dem_tiles</code> con datos; los <code>.geo</code> se leen.
<code>Paquete</code>	Atribución y zona declaradas, capa de arranque existente, ids no repetidos, y todo dentro de la carpeta (lo de fuera no viaja).

El modo **rápido** (sin cobertura) lo ejecuta `MapWidget` al abrir un paquete y deja los problemas en `dataWarnings()` ; el mapa arranca igual con lo que funcione. El completo lo usa la herramienta `check_data` .

8b · Módulo dem — elevación del terreno (HGT/SRTM)

Da la cota en metros de una coordenada, leyendo datos **SRTM**. Vive en `libmapa_core` (sin widgets): se consulta igual desde la ventana que desde consola o un test. Separa *de dónde salen las muestras* de *cómo se interpola*, con una interfaz al estilo de `ITileSource`:

- **IElevationSource** `src/dem/IElevationSource.h`: interfaz pura, un método `elevationAt(coord)→double` (contrato NaN).
- **GridElevation** `src/dem/GridElevation.h`: base que concentra lo común (caché LRU de tiles, lectura de muestra, **interpolación bilineal**, huecos→NaN); deja un solo hueco: `loadTile()`.
- **HgtElevation**: `loadTile()` lee el fichero `.hgt` de una carpeta.
- **SqliteElevation**: `loadTile()` consulta una base de datos `.sqlitedb` y descomprime el blob del tile — un **solo fichero**, cómodo para empaquetar en una app.

El `.hgt` es crudo y **sin cabecera**: una rejilla **cuadrada** de muestras `int16` **big-endian** que cubre $1^\circ \times 1^\circ$. El nombre marca la esquina suroeste (`N19W077.hgt` = 19–20 N, 77–76 O); la fila 0 es el borde **norte** y la columna 0 el **oeste**. La resolución no está escrita: se deduce del tamaño (`lado = isqrt(bytes/2)` → $1201 = 90$ m, $3601 = 30$ m).

Función	Qué hace
<code>elevationAt(coord) → double</code>	Localiza el tile (floor de lat/lon), autodetecta el lado, lee big-endian e interpola bilinealmente . NaN si no hay dato (tile ausente, sin origen, o hueco SRTM -32768).
<code>loadTile(lat,lon,...)</code>	El único método que cambia por origen: lee el fichero (<code>HgtElevation</code>) o el blob de la BD (<code>SqliteElevation</code>).

El `.hgt` es crudo y **sin cabecera**: rejilla **cuadrada** de `int16` **big-endian** de $1^\circ \times 1^\circ$, nombre = esquina SO (`N19W077.hgt`), fila 0 = norte, col 0 = oeste. La resolución se deduce del tamaño (`lado = isqrt(bytes/2)` → $1201 = 90$ m, $3601 = 30$ m). La **BD** guarda esas mismas muestras `qCompress` -adas:

`dem_tiles(lat,lon,side,data) + dem_meta`; se abre read-only por el `SqliteConnectionPool`.

Propiedades: `caché LRU` de tiles abiertos; `huecos→NaN` no inventa terreno; `dos orígenes` carpeta `.hgt` o BD empaquetable, misma cota. Se expone por la fachada: `MapConfig.elevationDir / elevationDbFile`, `MapWidget::elevationAt()` y `setElevationDir/Db()`; `fill_map` muestra la cota bajo el cursor (`--dem` carpeta, `--dem-db` base de datos, o el botón **DEM...**).

9 · Flujos de interacción entre objetos

9.1 · Pintar el mapa mientras navegas



Figura 2 — De un arrastre a la imagen en pantalla. El hilo lector nunca bloquea la interfaz; el caché evita releer.

9.2 · Objetivos móviles en tiempo real

`MapWidget::updateTarget(id, pos, rumbo) → TargetModel` actualiza posición y traza en $O(1)$ y emite `changed()` → `TargetLayer` **agrupa** los avisos y repinta como mucho ~30 veces/segundo, y **solo** su capa (modo BUFFERED), sin tocar teselas ni entidades. Así aguanta cientos de móviles.

9.3 · Entidades y ficheros .geo

`loadGeoAsLayer(ruta) → GeoFile` lee los trazados → `OverlayModel` crea una entidad multi-parte → `FeatureLayer` la pinta. `saveFeaturesTo()` vuelca todo con `VectorRepository`.

10 · Codificación de teselas (la matemática)

Cada BD guarda las teselas con su propio convenio. La librería lo captura en el `TileDataset` y traduce siempre igual. Dado un **tesela lógica** (z, x, y) en el estándar XYZ (origen arriba-izquierda):

```
z_guardado = zFactor * z + zOffset      (columna z de la tabla)
x_guardado = x                          (igual)
y_guardado = toStorageY(y, z, esquema)  XYZ: y | TMS: (2^z - 1) - y
s          = sValue                    (si la BD tiene columna 's')
```

BD	zFactor	zOffset	esquema	columna s	Ejemplo z lógico 11
OSM (Cuba)	1	1	XYZ	sí (2686452)	z_guardado = 12
Satelital (Google)	-1	17	XYZ	sí (0)	z_guardado = 6
Vector / base nueva	1	0	XYZ	no	z_guardado = 11

Este mapeo es **el mismo** que usan al leer el widget y al escribir las herramientas de descarga. Por eso una tesela bajada por `fill_tiles` aparece exactamente donde el mapa la espera.

11 · Herramientas desarrolladas

Programas de línea de comandos (y uno con mapa) que se apoyan en la librería.

Herramienta	Tipo	Para qué
probe_db	consola	Sonda una <code>.sqlitedb</code> y deduce su configuración; imprime el bloque para <code>datasets.json</code> .
geo_to_tiles	consola (Gui)	Rasteriza un vector <code>.geo/.xyz</code> a una pirámide de teselas SQLite (capa base rápida).
vector_db	consola	Utilidad sobre la BD de entidades vectoriales.
render_map	utilidad (widget)	Renderiza el mapa a imagen sin interacción (pruebas/verificación).
bench_tiles	consola	Mide el rendimiento de lectura/caché de teselas.
fill_tiles	consola	Descarga las teselas que faltan (o crea una base nueva) de una fuente XYZ sin clave.
fill_map	ventana + mapa	Lo mismo pero visual: marca el área en el mapa, rango de zoom, barra de progreso.
fill_hgt	consola	Descarga <code>.hgt</code> de 30 m de AWS Skadi (sin clave) para una zona (<code>--bbox / --cuba</code>); descomprime gzip con miniz vendoreado (sin zlib externa); reanudable. Alimenta a <code>dem_to_db</code> .
dem_to_db	consola	Construye la base de datos de elevación (<code>.sqlitedb</code>) desde una carpeta de <code>.hgt</code> (blobs <code>qCompress</code> , ~30 % del tamaño). La lee <code>SqliteElevation</code> .

Pipeline de elevación empaquetable: `fill_hgt --cuba` (baja 30 m sin clave) → `dem_to_db` (BD comprimida) → `fill_map --dem-db` o `MapWidget::elevationAt()` en tu app. El generador acepta también `.hgt` de 90 m ya existentes.

A continuación, la **síntesis de comando y los argumentos de entrada de cada herramienta**. Las opciones entre [...] son opcionales; el resto, obligatorias. Los bbox siempre son `latN,lonO,latS,lonE` (norte, oeste, sur, este).

probe_db — sondar BD y generar datasets.json / mapa.json

```
probe_db --id <id> --file <ruta.sqlitedb> [--id ... --file ...] [--ref-bbox latN,lonO,latS,lonE] [--out datasets.json] [--no-test] [--package [--dem ruta] [--overlay f.geo]... [--features nombre]]
```

<code>--id</code>	Identificador de la capa. Repetible ; en el mismo orden que los <code>--file</code> .
<code>--file</code>	Ruta al <code>.sqlitedb</code> a sondar. Repetible , pareado con <code>--id</code> .
<code>--name</code>	Nombre visible de la capa (opcional, repetible).
<code>--ref-bbox</code>	Rectángulo de referencia para desempatar esquema XYZ vs TMS. Cuba: <code>23.3,-85.0,19.7,-74.0</code> .
<code>--out</code>	Fichero JSON de salida (def. <code>datasets.json</code>). Las rutas se escriben relativas a su carpeta .
<code>--no-test</code>	No verificar que la fuente lee de verdad (solo deduce).

<code>--package</code>	Genera el manifiesto completo de paquete (<code>mapa.json v2</code>): <code>package</code> (id y nombre de la carpeta, fecha de hoy, zona del <code>--ref-bbox</code>), <code>start</code> y <code>features</code> .
<code>--dem</code>	Con <code>--package</code> : BD de elevación o carpeta de <code>.hgt</code> .
<code>--overlay</code>	Con <code>--package</code> : capa fija <code>.geo</code> (repetible).
<code>--features</code>	Con <code>--package</code> : nombre de la BD de entidades del usuario (def. <code>entidades.db</code>).

check_data — comprobar un paquete antes de distribuirlo

`check_data <carpeta_paquete | mapa.json> [--quick] [--max-zoom N] [--strict]`

<code>paquete</code>	Carpeta del paquete o su <code>mapa.json</code> .
<code>--quick</code>	Solo lo rápido (ficheros, apertura, decodificación); sin cobertura por zoom.
<code>--max-zoom</code>	No medir cobertura por encima de este zoom.
<code>--strict</code>	Salir con 1 también si hay avisos (por defecto solo con errores).

Imprime una ficha por capa (tamaño, formato de imagen y cobertura por zoom dentro de la zona) y los hallazgos de más a menos graves. Salida 0 = listo para distribuir, 1 = errores, 2 = argumentos mal; pensado para scripts.

geo_to_tiles — rasterizar vector a teselas

`geo_to_tiles --in <fichero.geo> --out <salida.sqlitedb> --id <id> --name "<nombre>" [--minzoom N] [--maxzoom N] [--color #hex] [--width f] [--fill] [--bg #hex]`

<code>--in</code>	Fichero de entrada <code>.geo</code> / <code>.xyz</code> (trazados separados por <code>0,0</code>).
<code>--out</code>	<code>.sqlitedb</code> de teselas a crear.
<code>--id / --name</code>	Identificador y nombre de la capa generada.
<code>--minzoom / --maxzoom</code>	Rango de zoom de la pirámide (def. 4..11).
<code>--color / --width</code>	Color y grosor de línea del trazado.
<code>--fill / --bg</code>	Rellenar polígonos / color de fondo.

vector_db — BD de entidades vectoriales

`vector_db --out <mapdata.db> [--dump] · vector_db --file <mapdata.db> --dump`

<code>--out</code>	Crea una BD nueva con datos de ejemplo (la borra si existe).
<code>--file</code>	Abre una BD existente sin tocarla (para inspección).
<code>--dump</code>	Vuelca esquema y contenido (implícito al crear con <code>--out</code>).

render_map — mapa a imagen (sin ventana)

`render_map (--datasets <datasets.json> | --data <carpeta_paquete>) --out <mapa.png> [--layer id] [--center lat,lon] [--zoom N] [--size AnchoxAlto] [--wait ms] [--grid] [--features fichero.geo]`

<code>--datasets / --out</code>	Config de capas y PNG de salida (obligatorios).
---------------------------------	---

<code>--data</code>	En vez de <code>--datasets</code> : abre un paquete (<code>mapa.json</code>); capa, centro y zoom salen de su <code>start</code> salvo que se pasen, y dibuja sus capas fijas.
<code>--layer</code>	Capa base a dibujar (def. la primera del JSON).
<code>--center</code> / <code>--zoom</code>	Centro <code>lat,lon</code> y nivel de zoom.
<code>--size</code>	Tamaño en píxeles, <code>1280x800</code> .
<code>--wait</code>	Milisegundos de espera a que terminen de cargar las teselas.
<code>--grid</code>	Dibuja la rejilla <code>z/x/y</code> (verde = propia, rojo = respaldo escalado).
<code>--features</code>	Superpone las entidades de un <code>.geo</code> .

bench_tiles — medir rendimiento de lectura

`bench_tiles --datasets <datasets.json> [--zoom N] [--width N] [--height N] [--center lat,lon] [--pans N]`

<code>--datasets</code>	Config de capas a medir (obligatorio).
<code>--zoom</code>	Nivel de zoom de la prueba.
<code>--width</code> / <code>--height</code>	Tamaño del viewport simulado.
<code>--center</code>	Centro <code>lat,lon</code> del barrido.
<code>--pans</code>	Número de desplazamientos a simular.

fill_hgt — descargar elevación SRTM (30 m)

`fill_hgt (--cuba | --bbox latN,lonO,latS,lonE) --out <carpeta> [--url base] [--res 30]`

<code>--cuba</code>	Preset: bbox de Cuba (lat 19..24, lon -85..-74).
<code>--bbox</code>	Zona a cubrir, <code>latN,lonO,latS,lonE</code> (alternativa a <code>--cuba</code>).
<code>--out</code>	Carpeta donde dejar los <code>.hgt</code> (reanudable: salta los que ya están).
<code>--url</code>	Base de la fuente (def. AWS Skadi <code>elevation-tiles-prod/skadi</code>).
<code>--res</code>	Resolución; hoy solo 30 m (Skadi). Se acepta y se ignora.

dem_to_db — carpeta .hgt → base de datos de elevación

`dem_to_db <carpeta_hgt> --out <dem.sqlitedb> [--overwrite]`

<code><carpeta_hgt></code>	Carpeta con los <code>.hgt</code> (30 m o 90 m; la resolución se autodetecta por tile). Argumento posicional.
<code>--out</code>	Base de datos <code>.sqlitedb</code> a crear (tablas <code>dem_tiles</code> + <code>dem_meta</code> , blobs <code>qCompress</code>).
<code>--overwrite</code>	Rehacer la BD si ya existe.

fill_tiles — argumentos

<code>--datasets</code> / <code>--id</code>	Rellenar una BD existente: de <code>datasets.json</code> toma codificación y ruta de la capa.
<code>--new</code> / <code>--name</code>	Crear una base nueva desde cero (codificación limpia XYZ, sin columna s).

<code>--bbox latN,lonO,latS,lonE</code>	Rectángulo a cubrir.
<code>--poly "lat,lon;lat,lon;..."</code>	Polígono (≥ 3 vértices): solo baja las teselas cuyo centro cae dentro. Alternativa a <code>--bbox</code> .
<code>--minzoom / --maxzoom</code>	Rango de zoom.
<code>--url</code>	Fuente XYZ (por defecto Esri «Clarity», sin clave).
<code>--rate / --conns</code>	Ritmo (lanzamientos/seg, tope suave) y peticiones en vuelo a la vez (1..8, def. 2).
<code>--retries / --timeout</code>	Reintentos por tesela y espera máxima por petición.
<code>--overwrite / --only-missing</code>	Rebajar todo / (def.) solo lo que falta.
<code>--yes</code>	No preguntar confirmación.

fill_map — cómo se usa

```
fill_map [datasets.json] [--dem <carpeta_hgt>] [--dem-db <dem.sqlitedb>]
```

El primer argumento posicional es el `datasets.json` (def. `./datasets.json`); `--dem` fija una carpeta de `.hgt` y `--dem-db` una base de datos de elevación. Todo lo demás se maneja desde la ventana:

- **Rellenar**: marca un rectángulo (o escribe el `bbox`) + rango de zoom → completa la capa activa.
- **Polígono**: marca un polígono clic a clic (doble clic / Enter lo cierra) y descarga solo las teselas de dentro. Excluyente con "Seleccionar área".
- **Nueva base...**: crea un `.sqlitedb` nuevo, todo de la fuente elegida, del área marcada.
- **Estimación de tamaño**: antes de confirmar, un muestreo rápido (`estimateSize`) dice cuántos MB ocupará, además del nº de teselas. Si no logra muestrear, da una cifra "aprox.".
- **Descargar al navegar**: mientras te mueves, baja lo que falte al zoom actual.
- Tres formas de indicar el área: dibujar rectángulo, escribir `bbox` , o la vista actual.
- **Rejilla**: dibuja el borde de cada tesela con su `z/x/y` (verde = propia, rojo = respaldo escalado) para ver qué falta a este zoom.
- **Cobertura** (+ selector de zoom): mancha fija que muestra qué zonas de ESE zoom ya están en la BD, visible aunque mires a un zoom menor; se refresca sola mientras descargas (ver `CoverageLayer`).
- **DEM...** (o `--dem <carpeta>`): carga ficheros SRTM `.hgt` y muestra la **cota del terreno bajo el cursor** en la barra de estado; "—" si no hay dato (ver módulo `dem`).

Orden de descarga (`TileFiller::advanceCursor`): por nivel de zoom de menor a mayor (termina un nivel antes de pasar al siguiente, emitiendo `zoomFinished`); dentro de cada nivel, columna a columna de oeste a este y cada columna de norte a sur; saltando las teselas que ya existen (salvo `--overwrite`). Es un barrido sistemático del `bbox`, reanudable.

12 · Motor de descarga `TileFiller`

El corazón común de `fill_tiles` y `fill_map` (`tools/common/TileFiller.h`). Es un `QObject` **asíncrono**: no bloquea, se apoya en el bucle de eventos (un temporizador marca el ritmo y cada respuesta encadena la siguiente). Así el mismo motor —y la misma codificación probada— sirve para la consola y para la ventana con barra de progreso viva.

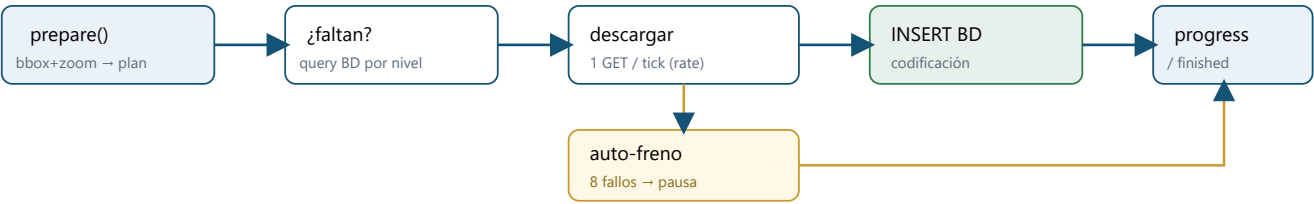


Figura 3 — Flujo de `TileFiller`: estima, descarga a ritmo, inserta con la codificación de la base y avisa; si hay muchos fallos seguidos, pausa y reanuda.

Función / señal	Qué hace
<code>prepare(params, err)</code>	Abre la BD (la crea si <code>createSchema</code>), calcula cuántas faltan por zoom. Sin red.
<code>estimateSize(n=12)</code>	Muestreo asíncrono: baja <code>n</code> teselas repartidas por zoom (solo mide bytes, no escribe BD) y emite <code>sizeEstimated</code> . Entre <code>prepare()</code> y <code>start()</code> .
<code>signal sizeEstimated(KiB, bytesTotal, muestras)</code>	Tamaño estimado $\approx$ media-total; <code>muestras=0</code> = heurística (sin red) $\sim$ 20 KiB/tesela.
<code>start()</code> / <code>cancel()</code>	Empieza / corta la descarga asíncrona.
<code>signal progress(hechas,total,t/s)</code>	Avance y velocidad media (barra + ETA).
<code>signal zoomFinished(z, nuevas)</code>	Un nivel terminado.
<code>signal throttling(seg, racha)</code>	Auto-freno : la fuente limita; pausa <code>seg</code> y reanuda.
<code>signal message / finished</code>	Motivo real de un fallo / fin con estadísticas.

Propiedades clave: `reanudable` solo baja lo que falta (se corta y se relanza sin repetir); `paralelo` hasta `connections` peticiones en vuelo a la vez (cada una con su `Pending` y su `timeout`), con `rate` como tope de lanzamientos/seg; `codificación exacta` inserta con `storedZ /Y/s` de la base; `polígono` descarta las teselas con centro fuera; `auto-freno` pausa con backoff creciente (30→60→120...→300 s) ante rachas de fallos y reanuda al primer éxito; `busy_timeout` convive con el mapa leyendo la misma BD.

13 · Decisiones de diseño e historia

- **Núcleo sin ventanas.** Separar `libmapa_core` de `libmapa_widget` permite probar el núcleo sin GUI (14 de los 17 tests) y no imponer `QCustomPlot`.
- **PIMPL en MapWidget.** `QCustomPlot` queda dentro; la cabecera pública es limpia.
- **Configuración en datos.** El mapeo de zoom, el esquema y las columnas viven en `datasets.json`, no cableados en el código.
- **Zoom hasta z15+.** `recommendedMaxZoom` separa «hasta dónde hay cobertura buena» de «hasta dónde hay teselas sueltas».
- **Compatibilidad Qt.** Objetivo actual **Qt 5.14 / 5.15 / 6.x**; el código es C++17 y usa guardas `QT_VERSION` solo donde Qt5 y Qt6 difieren de verdad (redirecciones de red, `qHash`, `QWheelEvent::position`).
- **Fuente por defecto Esri «Clarity».** Comparada contra la base Google, Clarity es la fuente gratis sin clave que más se acerca en color (menos costuras al rellenar huecos).
- **Descarga responsable.** Ritmo configurable y **auto-freno**: los servicios gratis limitan el uso masivo; conviene z altos solo por zonas y ritmo suave.

14 · Cómo compilar y usar

CMake (recomendado)

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
# Windows/MinGW: cmake -G "MinGW Makefiles" -S . -B build ...
```

El widget (MapWidget, fill_map, demo) requiere **QCustomPlot** en `third_party/qcustomplot/` (gitignored). Sin él, se compila solo el núcleo y las herramientas de consola, y CMake lo avisa.

qmake (alternativa)

```
qmake qmake/fill_tiles.pro && make      # o fill_map.pro / probe_db.pro
# el arranque de tu app:  qmake miapp.pro
```

Un flujo típico de datos

1. Preparar los datos (con internet): `fill_tiles` / `fill_map` para teselas, `fill_hgt` → `dem_to_db` para la elevación.
2. `probe_db --package` sobre la carpeta → genera su `mapa.json` (rutas relativas). Se ajusta a mano lo que haga falta (nombres, atribución, estilos).
3. `check_data` sobre la carpeta: 0 errores → el paquete está listo.
4. Se copia la carpeta junto a la aplicación. La app solo necesita `cfg.dataDir = ../datos`: el producto final **no usa internet**.

LibMapaStatic — documentación técnica generada a partir del código de la rama de trabajo. Núcleo (`geo · db · tiles · io`) + widget (QCustomPlot) + herramientas (`probe_db · geo_to_tiles · fill_tiles · fill_map`).